Étape 3 – Stratégies SCM et QA

Gestion du code source et assurance qualité : IDS Comportemental

Ce document définit la stratégie de gestion du contrôle de source (SCM) et d'assurance qualité (QA) qui encadrera le développement du MVP, en cohérence avec un travail en solo.

1. Stratégie SCM (gestion de version)

Stratégie de branches

Branche	Rôle
main	Toujours stable ; version démontrée au jury
dev	Branche d'intégration regroupant les fonctionnalités terminées
feature/xxx	Une branche par fonctionnalité, créée depuis dev

Exemples de branches de fonctionnalités prévues

feature/capture-scapy — feature/ml-isolation-forest — feature/api-flask — feature/dashboard-frontend — feature/api-abuseipdb

Workflow

1. Créer une branche feature/... depuis dev.
2. Développer et effectuer des commits réguliers.
3. Une fois la fonctionnalité testée, fusionner dans dev via une pull request.
4. À la fin d'un sprint, si dev est stable, fusionner dev dans main.

Convention de commits

Format : type: description

```
feat: ajout capture paquets Scapy
fix: correction calcul score anomalie
docs: mise a jour README
```

Revue de code avant fusion (checklist)

— Le code fonctionne-t-il sans erreur ?
— Est-il commenté et lisible ?
— Respecte-t-il une convention de nommage cohérente ?
— Aucune clé API ou mot de passe en dur dans le code ?

2. Stratégie QA (assurance qualité)

Tests manuels

Composant	Méthode de test
Capture réseau	Générer du trafic normal et anormal (ex : nmap sur la machine locale) et vérifier l'extraction correcte des métriques
Modèle ML	Vérifier la cohérence des scores d'anomalie sur des cas connus
API Flask	Tester chaque endpoint avec Postman ou curl (codes de retour, formats, cas d'erreur)
Dashboard web	Vérifier l'affichage des alertes, le fonctionnement des filtres et du bouton « Traiter »

Tests automatisés (pytest)

— Tests unitaires sur FeatureExtractor : le calcul des métriques est-il correct pour un paquet donné ?
— Tests unitaires sur AnomalyDetector : le modèle détecte-t-il une anomalie injectée dans un jeu de données de test ?
— Tests d'intégration sur les endpoints Flask, à l'aide du client de test intégré à Flask.

Suivi des bugs

Les bugs identifiés sont notés dans les Issues GitHub du dépôt, avec une étiquette de priorité : bug, critique ou mineur.